

Домашнее задание по ClickHouse

Агрегатные функции, функции для работы с типами данных и User-Defined Functions

Ниже приведены задания и SQL-решения. Скриншоты выполнения вынесены в приложение в конце документа.

1. Агрегатные функции

1.1. Рассчитайте общий доход от всех операций.

Доход с одной строки равен $\text{quantity} * \text{price}$, поэтому нужно суммировать это выражение по всей таблице.

```
SELECT SUM(price * quantity) AS revenue
FROM transactions;
```

1.2. Найдите средний доход с одной сделки.

Используется AVG по выражению $\text{quantity} * \text{price}$, потому что доход считается по каждой транзакции.

```
SELECT AVG(price * quantity) AS average
FROM transactions;
```

1.3. Определите общее количество проданной продукции.

Нужно просуммировать quantity по всем строкам.

```
SELECT SUM(quantity) AS sold_products
FROM transactions;
```

1.4. Подсчитайте количество уникальных пользователей, совершивших покупку.

Для этого используется COUNT(DISTINCT user_id).

```
SELECT COUNT(DISTINCT user_id) AS total_users
FROM transactions;
```

2. Функции для работы с типами данных

2.1. Преобразуйте transaction_date в строку формата YYYY-MM-DD.

Форматирование выполняется через formatDateTime.

```
SELECT formatDateTime(transaction_date, '%Y-%m-%d') AS formatted_date
FROM transactions;
```

2.2. Извлеките год и месяц из transaction_date.

Используются функции EXTRACT(YEAR ...) и EXTRACT(MONTH ...).

```
SELECT
    EXTRACT(YEAR FROM transaction_date) AS year,
    EXTRACT(MONTH FROM transaction_date) AS month
FROM transactions;
```

2.3. Округлите price до ближайшего целого числа.

Для округления используется round(price).

```
SELECT round(price) AS rounded_price
FROM transactions;
```

2.4. Преобразуйте transaction_id в строку.

Для преобразования в строку используется toString(transaction_id).

```
SELECT toString(transaction_id) AS transaction_id_str
FROM transactions;
```

3. User-Defined Functions (UDFs)

3.1. Создайте простую UDF для расчета общей стоимости транзакции.

Функция умножает price на quantity и возвращает стоимость одной транзакции.

```
CREATE FUNCTION sum_transaction_price AS (price, quantity) -> price * quantity;
```

3.2. Используйте созданную UDF для расчета общей цены для каждой транзакции.

Функция вызывается для каждой строки таблицы transactions.

```
SELECT sum_transaction_price(price, quantity) AS total_price  
FROM transactions;
```

3.3. Создайте UDF для классификации транзакций.

Классификация зависит от суммы price * quantity: меньше 100 - Budget, меньше 500 - Mid-range, иначе Premium.

```
CREATE FUNCTION qualify_transaction_via_price AS (price, quantity) ->  
    if(price * quantity < 100, 'Budget',  
        if(price * quantity < 500, 'Mid-range', 'Premium'));
```

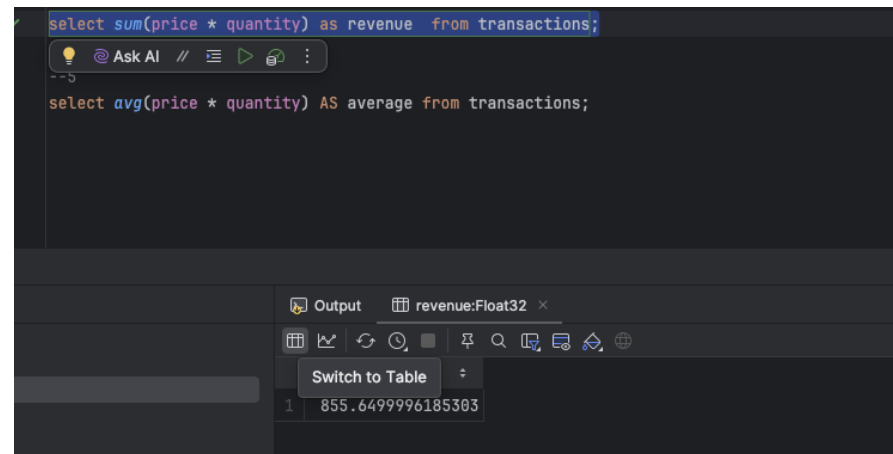
3.4. Примените UDF для категоризации каждой транзакции.

Функция вызывается для каждой строки и возвращает категорию транзакции.

```
SELECT qualify_transaction_via_price(price, quantity) AS transaction_category  
FROM transactions;
```

Приложение. Скриншоты выполнения

Скриншот 1. Общий доход от всех операций.

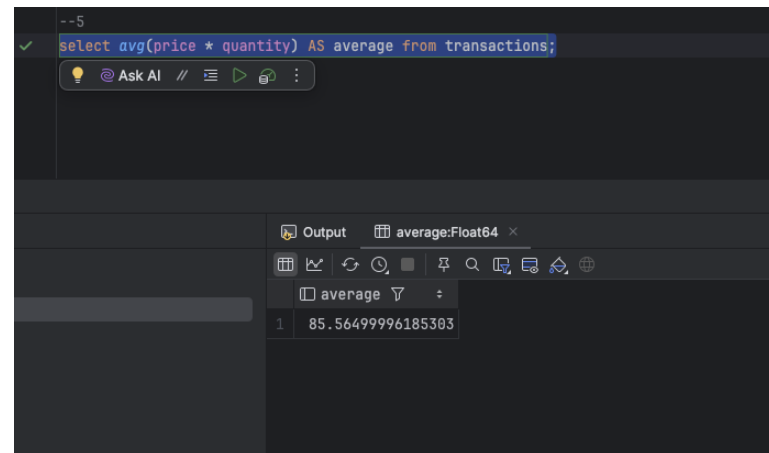


```
--5
select sum(price * quantity) as revenue from transactions;

--5
select avg(price * quantity) AS average from transactions;
```

revenue:Float32
855.6499996185303

Скриншот 2. Средний доход с одной сделки.

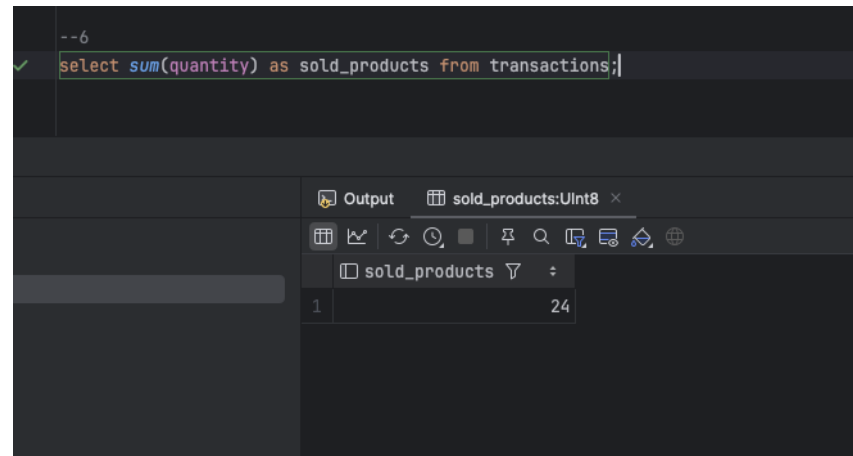


```
--5
select avg(price * quantity) AS average from transactions;
```

average:Float64
85.56499996185303

Скриншот 3. Общее количество проданной продукции.

```
--6
select sum(quantity) as sold_products from transactions;
```

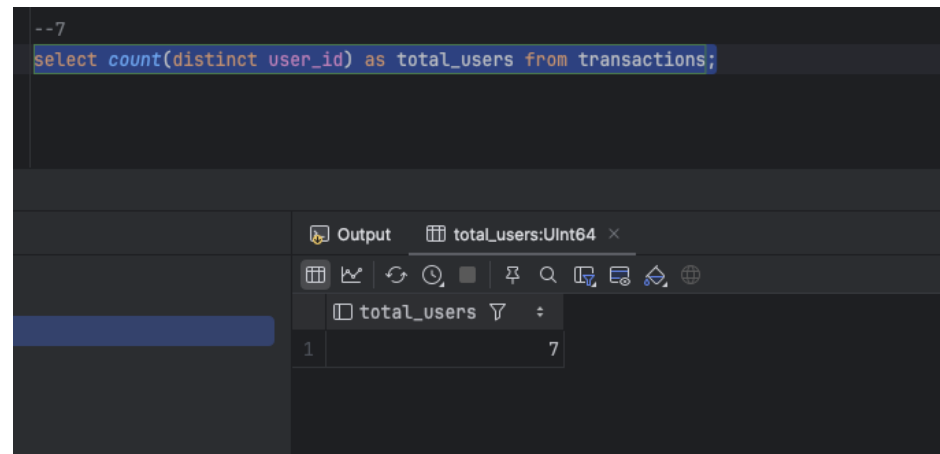


The screenshot shows a ClickHouse query interface. The query is `select sum(quantity) as sold_products from transactions;`. The output is displayed in a table with one row and one column, `sold_products`, with a value of 24.

sold_products
24

Скриншот 4. Количество уникальных пользователей.

```
--7
select count(distinct user_id) as total_users from transactions;
```



The screenshot shows a ClickHouse query interface. The query is `select count(distinct user_id) as total_users from transactions;`. The output is displayed in a table with one row and one column, `total_users`, with a value of 7.

total_users
7

Скриншот 5. Форматирование transaction_date в строку YYYY-MM-DD.

```
--8
SELECT formatDateTime(transaction_date, 'YY-MM-DD') AS formatted_date from transactions;

--9
SELECT toTypeName(formatDateTime(transaction_date, 'YY-MM-DD')) AS formatted_date from transactions;
```

Output formatted_date:String x

formatted_date
1 2024-01-01
2 2024-01-02
3 2024-01-03
4 2024-01-04
5 2024-01-05
6 2024-01-06
7 2024-01-07
8 2024-01-08
9 2024-01-09
10 2024-01-10

Скриншот 6. Извлечение года и месяца через EXTRACT.

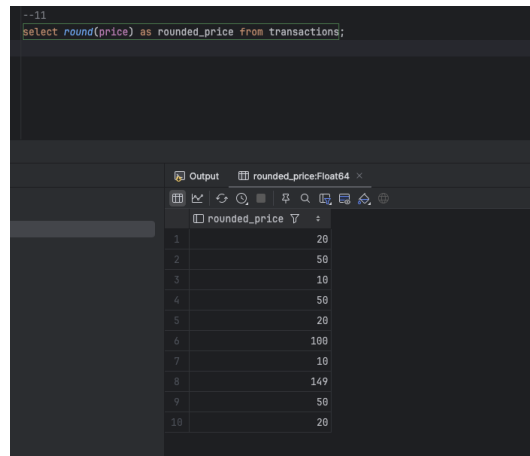
```
--10
SELECT
  EXTRACT(YEAR FROM transaction_date) AS year,
  EXTRACT(MONTH FROM transaction_date) AS month
FROM transactions
```

Output Result 35 x

year	month
1 2024	1
2 2024	1
3 2024	1
4 2024	1
5 2024	1
6 2024	1
7 2024	1
8 2024	1
9 2024	1
10 2024	1

Скриншот 7. Округление price до целого.

```
--11
select round(price) as rounded_price from transactions;
```

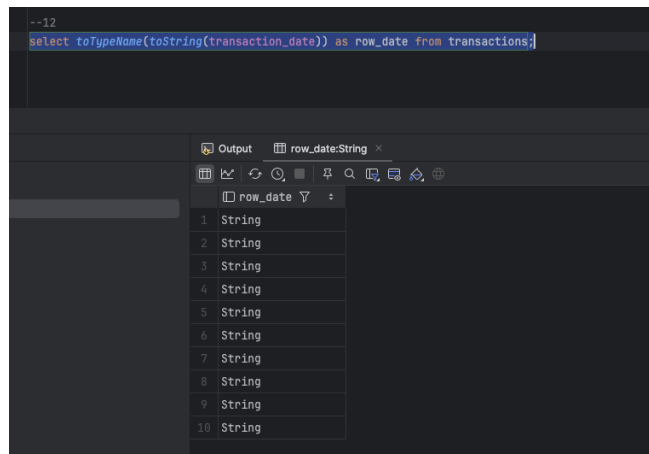


The screenshot shows the output of a SQL query in ClickHouse. The query is `select round(price) as rounded_price from transactions;`. The output is a table with 10 rows and 1 column named `rounded_price`. The data is as follows:

	rounded_price
1	20
2	50
3	10
4	50
5	20
6	100
7	10
8	149
9	50
10	20

Скриншот 8. Проверка строкового результата функции toString.

```
--12
select toTypeName(toString(transaction_date)) as row_date from transactions;
```



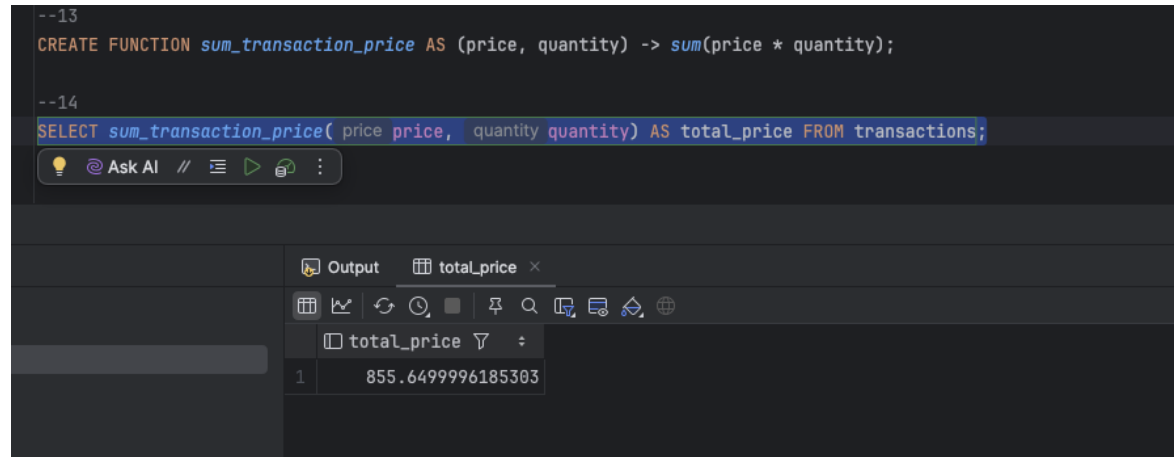
The screenshot shows the output of a SQL query in ClickHouse. The query is `select toTypeName(toString(transaction_date)) as row_date from transactions;`. The output is a table with 10 rows and 1 column named `row_date`. All values in the `row_date` column are `String`.

	row_date
1	String
2	String
3	String
4	String
5	String
6	String
7	String
8	String
9	String
10	String

Скриншот 9. UDF для общей стоимости транзакции.

```
--13
CREATE FUNCTION sum_transaction_price AS (price, quantity) -> sum(price * quantity);

--14
SELECT sum_transaction_price( price price, quantity quantity) AS total_price FROM transactions;
```



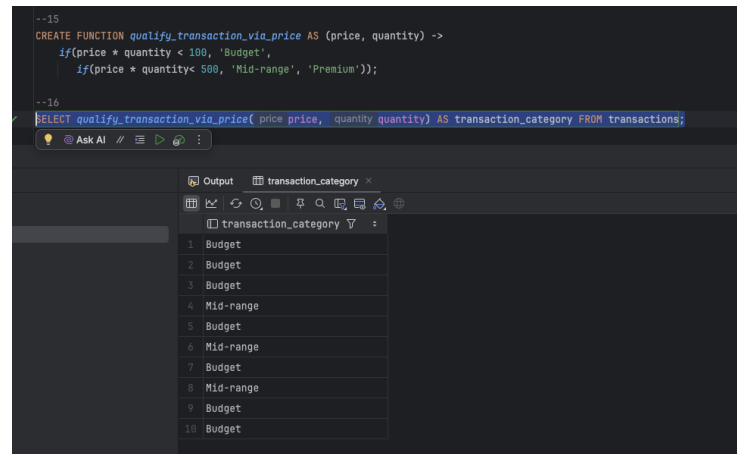
The screenshot shows a SQL IDE interface. The top panel contains SQL code for creating and using a UDF. The bottom panel shows the output of the query, which is a single row with the value 855.6499996185303.

total_price
855.6499996185303

Скриншот 10. UDF для классификации транзакций.

```
--15
CREATE FUNCTION qualify_transaction_via_price AS (price, quantity) ->
  if(price * quantity < 100, 'Budget',
    if(price * quantity < 500, 'Mid-range', 'Premium'));

--16
SELECT qualify_transaction_via_price( price price, quantity quantity) AS transaction_category FROM transactions;
```



The screenshot shows a SQL IDE interface. The top panel contains SQL code for creating and using a UDF. The bottom panel shows the output of the query, which is a list of transaction categories: Budget, Budget, Budget, Mid-range, Budget, Mid-range, Budget, Mid-range, Budget, Budget.

transaction_category
Budget
Budget
Budget
Mid-range
Budget
Mid-range
Budget
Mid-range
Budget
Budget